MesclaInvest: Firebase Functions e o desenvolvimento orientado a testes TDD (Test Driven Development)

Prof. Mateus Dias

Os códigos a seguir serão trabalhados na aula de hoje. Para um bom projeto de Firebase Functions (backend), devemos ter uma estrutura de diretórios e arquivos que facilite a evolução e a manutenção. Senão, quando o projeto cresce, novos requisitos surgem, se torna uma tarefa cansativa arrumar a casa depois de tanta coisa pronta.

Muitas equipes de projeto, começam um MVP (Produto Mínimo Viável) sem qualquer padrão, partindo do pressuposto: se gastarmos pouco tempo fazendo um MVP e não der certo, perdemos pouco tempo. Mas, quando o projeto recebe atenção dos investidores, engaja e traciona, a equipe técnica fica refém de uma arquitetura mal desenhada e difícil de escalar. Se tornando muito difícil transformar aquilo em produto.

Com pouco esforço é possível fazer o backend das functions de uma maneira boa, na medida certa e que possa escalar. O propósito dessa aula é aprender como fazer isso.

Quando usamos a abordagem TDD, desenvolvemos primeiro os testes integrando-os com a camada de negócios da aplicação, com o objetivo de validar os encaixes de integração e o funcional.

Muitas vezes é aí que o time ganha tempo. Enquanto uns fazem telas, layout etc, outros fazem backend e alguém orquestra e junta tudo. O TDD é excelente para isso.

Exemplo de código de testes em Flutter

widget_test.dart

Dart

```dart
import 'dart:convert';
import 'package:flutter_test/flutter_test.dart';
import 'package:http/http.dart' as http;

// para rodar o teste, basta:
// flutter test --dart-define=RUN_FIREBASE_FUNCTIONS_TESTS=true
const _runCallableTests = bool.fromEnvironment(
  'RUN_FIREBASE_FUNCTIONS_TESTS',
);

const _projectId = String.fromEnvironment(
  'FIREBASE_PROJECT_ID',
  defaultValue: '<coloque-id-projeto-firebase-aqui>',
);

// Emulador de Functions Local
const _functionsOrigin = String.fromEnvironment(
  'FIREBASE_FUNCTIONS_ORIGIN',
  defaultValue: 'http://127.0.0.1:5001',
);

// Emulador de Auth local
const _authOrigin = String.fromEnvironment(
  'FIREBASE_AUTH_ORIGIN',
  defaultValue: 'http://127.0.0.1:9099',
);


```

```
30 // Usuário de teste
31 const _testAuthEmail = 'mateus@mateus.pro.br';
32 const _testAuthPassword = '123456qwerty';
33
34 void main() {
35   group(
36     'Firebase callable functions de startups',
37     () {
38       late String idToken;
39
40       setUpAll(() async {
41         idToken = await _createAuthUserForTests();
42         await _callFunction('seedStartupCatalog');
43       });
44
45       test('seedStartupCatalog popula startups demonstrativas', () async {
46         final result = await _callFunction('seedStartupCatalog');
47         final data = result['data'] as Map<String, dynamic>;
48
49         expect(data['count'], 3);
50         expect(data['ids'], contains('biochip-campus'));
51         expect(data['ids'], contains('rota-verde'));
52         expect(data['ids'], contains('mentorai'));
53       });
54
55       test('listStartups lista e filtra startups autenticadas', () async {
56         final result = await _callFunction(
57           'listStartups',
58           idToken: idToken,
59           data: {'stage': 'nova', 'search': 'bio'},
60         );
61
62         final data = result['data'] as List<dynamic>;
63
64         expect(result['count'], 1);
65         expect(data.first['id'], 'biochip-campus');
66         expect(data.first['stage'], 'nova');
67       });
68
69       test('getStartupDetails retorna detalhes da startup', () async {
70         final result = await _callFunction(
71           'getStartupDetails',
72           idToken: idToken,
73           data: {'id': 'biochip-campus'},
74         );
75
76         final data = result['data'] as Map<String, dynamic>;
77         final access = data['access'] as Map<String, dynamic>;
78
79         expect(data['id'], 'biochip-campus');
80         expect(data['executiveSummary'], isNotEmpty);
81         expect(data['founders'], isNotEmpty);
82         expect(data['publicQuestions'], isA<List<dynamic>>());
83         expect(access['isInvestor'], isFalse);
84       });
85
86       test('createStartupQuestion cria pergunta publica', () async {
```

```
87          final result = await _callFunction(
88            'createStartupQuestion',
89            idToken: idToken,
90            data: {
91              'startupId': 'biochip-campus',
92              'text': 'Qual e o proximo marco do projeto?',
93              'visibility': 'publica',
94            },
95          );
96
97          final data = result['data'] as Map<String, dynamic>;
98
99          expect(data['id'], isNotEmpty);
100         expect(data['startupId'], 'biochip-campus');
101         expect(data['visibility'], 'publica');
102       });
103     },
104     skip: _runCallableTests ? false : _callableTestsSkipMessage,
105   );
106 }
107
108 const _callableTestsSkipMessage = 'Inicie os emuladores e rode com '
109     '--dart-define=RUN_FIREBASE_FUNCTIONS_TESTS=true.';
110
111 Uri _functionUri(String functionName) {
112   return Uri.parse(
113     '$_functionsOrigin/$_projectId/us-central1/$functionName',
114   );
115 }
116
117 Uri _authSignUpUri() {
118   return Uri.parse(
119     '$_authOrigin/identitytoolkit.googleapis.com/v1/'
120     'accounts:signUp?key=fake-api-key',
121   );
122 }
123
124 Uri _authSignInUri() {
125   return Uri.parse(
126     '$_authOrigin/identitytoolkit.googleapis.com/v1/'
127     'accounts:signInWithPassword?key=fake-api-key',
128   );
129 }
130
131 Future<String> _createAuthUserForTests() async {
132   final response = await http.post(
133     _authSignUpUri(),
134     headers: {'Content-Type': 'application/json'},
135     body: jsonEncode({
136       'email': _testAuthEmail,
137       'password': _testAuthPassword,
138       'returnSecureToken': true,
139     }),
140   );
141
142   final payload = _decodeResponse(response);
143
```

```
144   if (response.statusCode != 200 && _isEmailAlreadyInUse(payload)) {
145     return _signInAuthUserForTests();
146   }
147
148   if (response.statusCode != 200) {
149     fail('Falha ao criar usuario no Auth emulator: $payload');
150   }
151
152   return payload['idToken'] as String;
153 }
154
155 Future<String> _signInAuthUserForTests() async {
156   final response = await http.post(
157     _authSignInUri(),
158     headers: {'Content-Type': 'application/json'},
159     body: jsonEncode({
160       'email': _testAuthEmail,
161       'password': _testAuthPassword,
162       'returnSecureToken': true,
163     }),
164   );
165
166   final payload = _decodeResponse(response);
167
168   if (response.statusCode != 200) {
169     fail('Falha ao autenticar usuario no Auth emulator: $payload');
170   }
171
172   return payload['idToken'] as String;
173 }
174
175 bool _isEmailAlreadyInUse(Map<String, dynamic> payload) {
176   final error = payload['error'];
177
178   if (error is! Map<String, dynamic>) {
179     return false;
180   }
181
182   return error['message'] == 'EMAIL_EXISTS';
183 }
184
185 Future<Map<String, dynamic>> _callFunction(
186   String functionName, {
187   Map<String, dynamic> data = const {},
188   String? idToken,
189 }) async {
190   final headers = <String, String>{
191     'Content-Type': 'application/json',
192   };
193
194   if (idToken != null) {
195     headers['Authorization'] = 'Bearer $idToken';
196   }
197
198   final response = await http.post(
199     _functionUri(functionName),
200     headers: headers,
```

```
    body: jsonEncode({'data': data}),
  );

  final payload = _decodeResponse(response);

  if (response.statusCode != 200) {
    fail('Callable $functionName falhou: $payload');
  }

  if (payload['error'] != null) {
    fail('Callable $functionName retornou erro: ${payload['error']}');
  }

  return payload['result'] as Map<String, dynamic>;
}

Map<String, dynamic> _decodeResponse(http.Response response) {
  final decoded = jsonDecode(response.body);

  if (decoded is Map<String, dynamic>) {
    return decoded;
  }

  fail('Resposta inesperada: ${response.body}');
}
```

—

Firebase Functions

Agora, precisamos entender como a estrutura do Backend deve ser organizada.

Observe a estrutura de diretórios de um projeto Firebase functions, gerado pelo comando **firebase init**. Para cada domínio (tema), é importante criar essa estrutura para backend.

Considerando que tudo sobre startups no MesclaInvest ficaria no diretório startups, o diretório exchange seria o balcão de negociação de tokens.

text

```
projeto/
├── functions/
|   └── src/
|       └── startups/
|       |   ├── handlers/
|       |   ├── repositories/
|       |   ├── shared
|       |   └── types/
|       |   └── index.ts // exports do módulo startups
|       └── exchange/
|       |   ├── handlers/
|       |   ├── repositories/
|       |   ├── shared
|       |   └── types/
|       |   └── index.ts // exports do módulo exchange
|       └── index.ts // export final para o firebase
```

Portanto, o diretório **startups** quer dizer que teremos nele tudo do backend que cuida de startups: obter as startups, enviar perguntas para os sócios, consultar dados, participações de cada sócio etc. Tudo isso fica no domínio deste tema. Nada misturado e tudo organizado.

handlers: Nesse diretório estão as firebase functions. Uma por arquivo, nada de juntar tudo em um arquivo gigante.
repositories: Toda comunicação com o Firebase Firestore, é via repositório de dados. Por isso temos um padrão para manter isso separado das functions em si (mas eles prestam serviço às firebase functions - são usados nos handlers).
shared: arquivos em comum compartilhados entre os diretórios.
types: tipos específicos deste contexto (startups), type, interface, classes.
arquivo index.ts: exportação do módulo.

Vamos começar entendendo o que temos nos tipos (diretório types)

functions/src/startups/types/index.ts

TypeScript

```typescript
import {FieldValue, Timestamp} from "firebase-admin/firestore";

/**
  * Representa os estágios de maturidade aceitos para uma startup.
  *
  * Esses valores implementam a classificação descrita no item 5.2:
  *
  * - `nova`: ideia recentemente publicada.
  * - `em_operacao`: projeto já operando ou em validação prática.
  * - `em_expansao`: projeto em crescimento, com maior maturidade simulada.
  *
  * Os valores usam snake_case para facilitar armazenamento, filtros e
  * comparação direta no Firestore e nas chamadas callable.
  */
export type StartupStage = "nova" | "em_operacao" | "em_expansao";

/**
  * Define o nível de visibilidade de uma pergunta enviada à startup.
  *
  * - `publica`: aparece na área pública de perguntas e respostas da startup.
  * - `privada`: fica restrita a investidores, conforme regra do item 5.2.
  */
export type QuestionVisibility = "publica" | "privada";

/**
  * Dados mínimos do usuário autenticado necessários para regras de negócio.
  *
  * Este tipo é derivado do `request.auth` das Callable Functions. Ele evita que
  * os handlers dependam diretamente do formato completo do token Firebase e
  * preserva apenas o que o domínio precisa: UID e e-mail, quando disponível.
  */
export type AuthenticatedUser = {
  uid: string;
  email?: string;
};

/**
  * Representa um sócio, fundador ou participação societária da startup.
  *
  * Este tipo atende ao requisito de exibir estrutura societária e percentual de
  * participação dos sócios. Também permite registrar reservas e pools de
  * incentivo como linhas da composição societária simulada.
  */
export type Founder = {
```

```
  name: string;
  role: string;
  equityPercent: number;
  bio?: string;
};

/**
  * Representa conselheiros, mentores ou participantes externos.
  *
  * O item 5.2 pede que a aplicação exiba membros do conselho, mentores ou
  * pessoas externas quando aplicável. Ele modela essas participações sem
  * misturá-las com a estrutura societária.
  */
export type ExternalMember = {
  name: string;
  role: string;
  organization?: string;
};

/**
  * Documento completo de uma startup no Firestore.
  *
  * Este é o contrato principal do catálogo. Ele concentra os dados para
  * listagem, página detalhada, apresentação dos sócios, capital simulado e
  * materiais públicos do projeto.
  */
export type StartupDocument = {
  name: string;
  stage: StartupStage;
  shortDescription: string;
  description: string;
  executiveSummary: string;
  capitalRaisedCents: number;
  totalTokensIssued: number;
  currentTokenPriceCents: number;
  founders: Founder[];
  externalMembers: ExternalMember[];
  demoVideos: string[]; // poderia ser enderecos URLs no Youtube ou firebase storage.
  pitchDeckUrl?: string;
  coverImageUrl?: string;
  tags: string[];
  createdAt?: Timestamp;
  updatedAt?: Timestamp;
};

/**
  * Documento de pergunta armazenado na subcoleção da startup.
  *
  * As perguntas ficam em `startups/{startupId}/questions/{questionId}` para
  * manter o histórico associado ao projeto. A resposta é opcional porque a
  * pergunta pode ser criada antes de alguém respondê-la.
  */
export type StartupQuestionDocument = {
  authorUid: string;
  authorEmail?: string;
  text: string;
  visibility: QuestionVisibility;
```

```
  answer?: string;
  answeredAt?: Timestamp;
  createdAt: FieldValue;
};

/**
  * Versão resumida de startup usada na listagem do catálogo.
  *
  * Este tipo evita enviar todos os dados da startup para telas que precisam
  * apenas de cards ou linhas de lista. A tela detalhada deve usar
  * `StartupDocument`, mas a listagem usa `StartupListItem` para trafegar menos
  * dados e manter o contrato mais claro.
  */
export type StartupListItem = {
  id: string;
  name: string;
  stage: StartupStage;
  shortDescription: string;
  capitalRaisedCents: number;
  totalTokensIssued: number;
  currentTokenPriceCents: number;
  coverImageUrl?: string;
  tags: string[];
};
```

—

functions/src/startups/handlers/listStartups.ts

TypeScript

```
import {HttpsError, onCall} from "firebase-functions/https";
import {allowedStages} from "../shared/constants";
import {requireAuthenticatedUser} from "../shared/auth";
import {normalizeString} from "../shared/validation";
import {listStartupItems} from "../repositories/startupRepository";
import {StartupStage} from "../types";

/**
  * Lista as startups cadastradas no catálogo do MesclaInvest.
  *
  * Esta Function é callable porque será consumida diretamente pelo app mobile.
  * O app pode enviar, em `data`, os campos:
  *
  * - `stage`: filtro opcional por estágio.
  * - `search`: texto opcional para buscar no catálogo.
  *
  * A função exige usuário autenticado e retorna um objeto com:
  *
  * - `count`: quantidade de startups retornadas.
  * - `filters`: filtros aplicados e estágios disponíveis.
  * - `data`: lista resumida de startups para uso em telas de catálogo.
  */
export const listStartups = onCall(async (request) => {
  requireAuthenticatedUser(request);

  const stage = normalizeString(request.data?.stage);

  const search = normalizeString(request.data?.search)
```

```
    ?.toLocaleLowerCase("pt-BR");

  if (stage && !allowedStages.includes(stage as StartupStage)) {
    throw new HttpsError(
      "invalid-argument",
      "Filtro stage invalido. Use nova, em_operacao ou em_expansao."
    );
  }

  const startups = (await listStartupItems())
    .filter((startup) => !stage || startup.stage === stage)
    .filter((startup) => {
      if (!search) {
        return true;
      }

      const searchable = [
        startup.name,
        startup.shortDescription,
        startup.stage,
        ...startup.tags,
      ].join(" ").toLocaleLowerCase("pt-BR");

      return searchable.includes(search);
    })
    .sort((left, right) => left.name.localeCompare(right.name, "pt-BR"));

  return {
    count: startups.length,
    filters: {
      availableStages: allowedStages,
      stage: stage ?? null,
      search: search ?? null,
    },
    data: startups,
  };
});
```

—

functions/src/startups/handlers/createStartupQuestion.ts

TypeScript

```
import {FieldValue} from "firebase-admin/firestore";

import {HttpsError, onCall} from "firebase-functions/https";

import * as logger from "firebase-functions/logger";

import {allowedVisibilities} from "../shared/constants";

import {requireAuthenticatedUser} from "../shared/auth";

import {normalizeString} from "../shared/validation";

import {
  createQuestion,
  getStartupById,
```

```
16   userIsInvestor,
17 } from "../repositories/startupRepository";
18
19 import {QuestionVisibility, StartupQuestionDocument} from "../types";
20
21 /**
22   * Cria uma pergunta para uma startup.
23   *
24   * Esta Firebase Function é callable e deve ser chamada pelo app com:
25   *
26   * - `startupId`: identificador da startup.
27   * - `text`: texto da pergunta.
28   * - `visibility`: visibilidade opcional (`publica` ou `privada`).
29   *
30   * Perguntas públicas podem ser enviadas por qualquer usuário autenticado.
31   * Perguntas privadas exigem que o usuário tenha um documento em:
32   * `startups/{startupId}/investors/{uid}`.
33   */
34 export const createStartupQuestion = onCall(async (request) => {
35   const user = requireAuthenticatedUser(request);
36
37   const startupId = normalizeString(request.data?.startupId);
38
39   const text = normalizeString(request.data?.text);
40
41   const visibility = normalizeString(request.data?.visibility) ?? "publica";
42
43   if (!startupId || !text) {
44     throw new HttpsError("invalid-argument", "Informe startupId e text.");
45   }
46
47   if (!allowedVisibilities.includes(visibility as QuestionVisibility)) {
48     throw new HttpsError(
49       "invalid-argument",
50       "Visibility invalida. Use publica ou privada."
51     );
52   }
53
54   const startup = await getStartupById(startupId);
55
56   if (!startup) {
57     throw new HttpsError("not-found", "Startup nao encontrada.");
58   }
59
60   if (visibility === "privada") {
61     const isInvestor = await userIsInvestor(startupId, user.uid);
62
63     if (!isInvestor) {
64       throw new HttpsError(
65         "permission-denied",
66         "Somente investidores desta startup podem enviar perguntas privadas."
67       );
68     }
69   }
70
71   const question: StartupQuestionDocument = {
72     authorUid: user.uid,
```

```
    authorEmail: user.email,
    text,
    visibility: visibility as QuestionVisibility,
    createdAt: FieldValue.serverTimestamp(),
  };

  const questionId = await createQuestion(startupId, question);

  logger.info("Pergunta criada para startup.", {
    startupId,
    questionId,
    visibility,
  });

  return {
    data: {
      id: questionId,
      startupId,
      visibility,
    },
  };
});
```

—

functions/src/startups/handlers/getStartupDetails.ts

TypeScript

```
import {HttpsError, onCall} from "firebase-functions/https";

import {requireAuthenticatedUser} from "../shared/auth";

import {normalizeString} from "../shared/validation";

import {
  getStartupById,
  listPublicQuestions,
  userIsInvestor,
} from "../repositories/startupRepository";

/**
  * Busca os dados completos de uma startup específica.
  *
  * Esta Firebase Function é callable e deve ser chamada pelo app com:
  *
  * - `id`: identificador da startup no Firestore.
  *
  * A função exige autenticação e retorna a visão detalhada do item 5.2:
  * sumário executivo, estrutura societária, membros externos, vídeos,
  * perguntas públicas e flags de acesso para investidores.
  */
export const getStartupDetails = onCall(async (request) => {
  const user = requireAuthenticatedUser(request);

  const startupId = normalizeString(request.data?.id);

  if (!startupId) {
    throw new HttpsError(
```

```
      "invalid-argument",
      "Informe o parametro id da startup."
    );
  }

  const startup = await getStartupById(startupId);

  if (!startup) {
    throw new HttpsError("not-found", "Startup nao encontrada.");
  }

  const isInvestor = await userIsInvestor(startupId, user.uid);

  const questions = await listPublicQuestions(startupId);

  return {
    data: {
      id: startupId,
      ...startup,
      createdAt: startup.createdAt?.toDate().toISOString() ?? null,
      updatedAt: startup.updatedAt?.toDate().toISOString() ?? null,
      publicQuestions: questions,
      access: {
        isInvestor,
        canTradeTokens: isInvestor,
        canSendPrivateQuestions: isInvestor,
      },
    },
  };
});
```

—

functions/src/startups/handlers/seedStartupCatalog.ts

TypeScript

```
import {HttpsError, onCall} from "firebase-functions/https";
import {seedDemoStartups} from "../repositories/startupRepository";
import {normalizeString} from "../shared/validation";

/**
  * Popula o catálogo com startups demonstrativas.
  *
  * Esta Function é callable para facilitar a execução pelo app ou pelo
  * emulador durante desenvolvimento. Em ambiente de emulator ela roda sem chave.
  * Fora do emulator, exige `seedKey` em `request.data.seedKey`, comparando com a
  * variável de ambiente `SEED_STARTUP_CATALOG_KEY`.
  *
  * A função retorna a quantidade de startups gravadas e os ids dos documentos.
  */
export const seedStartupCatalog = onCall(async (request) => {
  if (!process.env.FUNCTIONS_EMULATOR) {
    const seedKey = normalizeString(request.data?.seedKey);

    if (!process.env.SEED_STARTUP_CATALOG_KEY ||
      seedKey !== process.env.SEED_STARTUP_CATALOG_KEY) {
      throw new HttpsError(
        "permission-denied",
```

```
        "Seed bloqueado fora do emulator sem seedKey valido."
      );
    }
  }

  const startupIds = await seedDemoStartups();

  return {
    data: {
      count: startupIds.length,
      ids: startupIds,
    },
  };
});
```

—

functions/src/startups/repositories/startupRepository.ts

TypeScript

```
import {FieldValue} from "firebase-admin/firestore";
import {
  StartupDocument,
  StartupListItem,
  StartupQuestionDocument,
} from "../types";
import {db} from "../shared/firebase";

const startupsCollection = db.collection("startups");

const demoStartups: Array<StartupDocument & {id: string}> = [
  {
    id: "biochip-campus",
    name: "BioChip Campus",
    stage: "nova",
    shortDescription: "Sensores portateis para analises laboratoriais " +
      "didaticas.",
    description: "A BioChip Campus simula kits de diagnostico rapido para " +
      "laboratorios universitarios, conectando sensores de baixo custo a um " +
      "aplicativo de acompanhamento.",
    executiveSummary: "Startup em fase de ideacao com foco em prototipagem " +
      "de sensores educacionais e validacao com cursos da area de saude.",
    capitalRaisedCents: 1850000,
    totalTokensIssued: 100000,
    currentTokenPriceCents: 125,
    founders: [
      {
        name: "Ana Ribeiro",
        role: "CEO",
        equityPercent: 48,
        bio: "Responsavel por estrategia e parcerias academicas.",
      },
      {
        name: "Lucas Moreira",
        role: "CTO",
        equityPercent: 37,
        bio: "Responsavel por hardware e integracao mobile.",
      },
```

```
      {name: "Mescla Labs", role: "Reserva estrategica", equityPercent: 15},
    ],
    externalMembers: [
      {
        name: "Dra. Helena Costa",
        role: "Mentora",
        organization: "PUC-Campinas",
      },
    ],
    demoVideos: ["https://example.com/videos/biochip-campus-demo"],
    pitchDeckUrl: "https://example.com/decks/biochip-campus.pdf",
    coverImageUrl: "https://images.unsplash.com/photo-" +
      "1581093458791-9d15482442f6",
    tags: ["healthtech", "iot", "educacao"],
  },
  {
    id: "rota-verde",
    name: "Rota Verde",
    stage: "em_operacao",
    shortDescription: "Otimizacao de rotas sustentaveis para entregas urbanas.",
    description: "A Rota Verde usa dados de distancia, emissao estimada e " +
      "ocupacao de entregadores para sugerir rotas urbanas com menor impacto " +
      "ambiental.",
    executiveSummary: "Startup em operacao piloto com pequenos comercios " +
      "locais e validacao de indicadores de economia de combustivel.",
    capitalRaisedCents: 7400000,
    totalTokensIssued: 250000,
    currentTokenPriceCents: 310,
    founders: [
      {name: "Beatriz Santos", role: "CEO", equityPercent: 42},
      {name: "Rafael Almeida", role: "COO", equityPercent: 28},
      {name: "Carla Nogueira", role: "CTO", equityPercent: 20},
      {name: "Reserva de incentivos", role: "Pool", equityPercent: 10},
    ],
    externalMembers: [
      {name: "Marcos Lima", role: "Conselheiro", organization: "Mescla"},
      {
        name: "Patricia Gomes",
        role: "Mentora",
        organization: "Rede de Logistica",
      },
    ],
    demoVideos: ["https://example.com/videos/rota-verde-demo"],
    pitchDeckUrl: "https://example.com/decks/rota-verde.pdf",
    coverImageUrl: "https://images.unsplash.com/photo-" +
      "1500530855697-b586d89ba3ee",
    tags: ["logtech", "sustentabilidade", "mobilidade"],
  },
  {
    id: "mentorai",
    name: "MentorAI",
    stage: "em_expansao",
    shortDescription: "Triagem inteligente para programas de mentoria " +
      "universitarios.",
    description: "A MentorAI organiza perfis de estudantes e mentores para " +
      "recomendar encontros com base em objetivos, disponibilidade e " +
      "historico de acompanhamento.",
```

```
96      executiveSummary: "Startup em expansao com uso simulado em programas de " +
97        "pre-aceleracao e potencial de integracao a plataformas educacionais.",
98      capitalRaisedCents: 12350000,
99      totalTokensIssued: 500000,
100     currentTokenPriceCents: 525,
101     founders: [
102       {name: "Diego Martins", role: "CEO", equityPercent: 36},
103       {name: "Juliana Vieira", role: "CPO", equityPercent: 24},
104       {name: "Felipe Andrade", role: "CTO", equityPercent: 25},
105       {
106         name: "Investidores simulados",
107         role: "Participacao externa",
108         equityPercent: 15,
109       },
110     ],
111     externalMembers: [
112       {
113         name: "Sofia Pereira",
114         role: "Conselheira",
115         organization: "Ecossistema Mescla",
116       },
117     ],
118     demoVideos: ["https://example.com/videos/mentorai-demo"],
119     pitchDeckUrl: "https://example.com/decks/mentorai.pdf",
120     coverImageUrl: "https://images.unsplash.com/photo-1552664730-d307ca884978",
121     tags: ["edtech", "ia", "mentoria"],
122   },
123 ];
124
125 function toListItem(id: string, startup: StartupDocument): StartupListItem {
126   return {
127     id,
128     name: startup.name,
129     stage: startup.stage,
130     shortDescription: startup.shortDescription,
131     capitalRaisedCents: startup.capitalRaisedCents,
132     totalTokensIssued: startup.totalTokensIssued,
133     currentTokenPriceCents: startup.currentTokenPriceCents,
134     coverImageUrl: startup.coverImageUrl,
135     tags: startup.tags,
136   };
137 }
138
139 export async function listStartupItems(): Promise<StartupListItem[]> {
140   const snapshot = await startupsCollection.limit(100).get();
141
142   return snapshot.docs.map((doc) =>
143     toListItem(doc.id, doc.data() as StartupDocument)
144   );
145 }
146
147 export async function getStartupById(
148   startupId: string
149 ): Promise<StartupDocument | undefined> {
150   const startupSnapshot = await startupsCollection.doc(startupId).get();
151
152   if (!startupSnapshot.exists) {
```

```
153     return undefined;
154   }
155
156   return startupSnapshot.data() as StartupDocument;
157 }
158
159 export async function userIsInvestor(
160   startupId: string,
161   uid: string
162 ): Promise<boolean> {
163   const investorSnapshot = await startupsCollection
164     .doc(startupId)
165     .collection("investors")
166     .doc(uid)
167     .get();
168
169   return investorSnapshot.exists;
170 }
171
172 export async function listPublicQuestions(startupId: string) {
173   const questionsSnapshot = await startupsCollection
174     .doc(startupId)
175     .collection("questions")
176     .where("visibility", "==", "publica")
177     .limit(50)
178     .get();
179
180   return questionsSnapshot.docs
181     .map((doc) => ({
182       id: doc.id,
183       text: doc.get("text"),
184       answer: doc.get("answer") ?? null,
185       answeredAt: doc.get("answeredAt")?.toDate?.()?.toISOString?.() ?? null,
186       createdAt: doc.get("createdAt")?.toDate?.()?.toISOString?.() ?? null,
187     }))
188     .sort((left, right) => String(right.createdAt ?? "")
189       .localeCompare(String(left.createdAt ?? "")));
190 }
191
192 export async function createQuestion(
193   startupId: string,
194   question: StartupQuestionDocument
195 ): Promise<string> {
196   const questionRef = await startupsCollection
197     .doc(startupId)
198     .collection("questions")
199     .add(question);
200
201   return questionRef.id;
202 }
203
204 export async function seedDemoStartups(): Promise<string[]> {
205   const batch = db.batch();
206
207   for (const startup of demoStartups) {
208     const {id, ...data} = startup;
209     const startupRef = startupsCollection.doc(id);
```

```

    batch.set(startupRef, {
      ...data,
      createdAt: FieldValue.serverTimestamp(),
      updatedAt: FieldValue.serverTimestamp(),
    }, {merge: true});
  }

  await batch.commit();

  return demoStartups.map((startup) => startup.id);
}
```

—

functions/src/startups/shared/auth.ts

TypeScript

```
import {CallableRequest, HttpsError} from "firebase-functions/https";
import {AuthenticatedUser} from "../types";

export function requireAuthenticatedUser(
  request: CallableRequest
): AuthenticatedUser {
  if (!request.auth) {
    throw new HttpsError(
      "unauthenticated",
      "Usuario precisa estar autenticado para acessar esta funcao."
    );
  }

  return {
    uid: request.auth.uid,
    email: request.auth.token.email as string | undefined,
  };
}
```

—

functions/src/startups/shared/constants.ts

TypeScript

```
import {QuestionVisibility, StartupStage} from "../types";

export const allowedStages: StartupStage[] = [
  "nova",
  "em_operacao",
  "em_expansao",
];

export const allowedVisibilities: QuestionVisibility[] = [
  "publica",
  "privada",
];
```

—

functions/src/startups/shared/firebase.ts

TypeScript

```typescript
import {getAuth} from "firebase-admin/auth";
import {getApps, initializeApp} from "firebase-admin/app";
import {getFirestore} from "firebase-admin/firestore";

if (getApps().length === 0) {
  initializeApp();
}

export const auth = getAuth();
export const db = getFirestore();
```

—

functions/src/startups/shared/validation.ts

TypeScript

```typescript
export function normalizeString(value: unknown): string | undefined {
  if (typeof value !== "string") {
    return undefined;
  }

  const trimmed = value.trim();
  return trimmed.length > 0 ? trimmed : undefined;
}
```

—

Exportações (módulo startups e total)

Ao final de construir o módulo startups, basta exportar todos os handlers (cada function).

functions/src/startups/index.ts

TypeScript

```typescript
export {createStartupQuestion} from "./handlers/createStartupQuestion";
export {getStartupDetails} from "./handlers/getStartupDetails";
export {listStartups} from "./handlers/listStartups";
export {seedStartupCatalog} from "./handlers/seedStartupCatalog";
```

—

Para o firebase, precisamos propagar os módulos completos desta maneira:

functions/src/index.ts

TypeScript

```typescript
import {setGlobalOptions} from "firebase-functions";

setGlobalOptions({maxInstances: 10});

export * from "./startups";
```

—